



## Proyecto Final

Prof Carmen Lucia Bustillo Hernández  
carmenbustillohernandez@gmail.com



Facultad de Ingeniería  
**Asignatura: Sistemas Distribuidos**  
Universidad Nacional Autónoma de México  
Fecha de Entrega: 21 de Mayo de 2026

**Nombre:** Enrique Yoab Cortez Rios

**Matrícula:** 317302992

**Nombre:** Mario Mendoza Gonzáles

**Matrícula:** 316284828

### Objetivo General

El alumno comprenderá y aplicará los mecanismos de intercomunicación de procesos y concurrencia a bajo nivel mediante el desarrollo de una aplicación distribuida peer-to-peer con interfaz gráfica.

### Objetivos Específicos

1. Diseñar e implementar una aplicación tipo chat (messenger) multiplataforma utilizando programación visual.

### Material

1. Equipo de cómputo con sistema operativo Windows. Mac o Linux y acceso a Internet.
2. Editor de textos (MS Word, Libre Office).
3. Compilador o IDE de lenguajes de programación.

### Bibliografía

1. Tanenbaum, A. S. (2004). Sistemas operativos modernos. En Prentice-Hall, Inc eBooks .  
<http://dl.acm.org/citation.cfm?id=120575>

## 1. Instrucciones Generales

1. Realizar el proyecto con las indicaciones que se solicitan.
2. Para el proyecto, desarrolle su diagrama de flujo y pseudocódigo respectivamente.

## 2. Criterios de Evaluación

1. Se evaluará de acuerdo con los Criterios de Evaluación que se encuentra en el Google Classroom de la asignatura.

## 3. Observaciones Previas

- **Número de Procesos:** El sistema requiere la bifurcación de dos procesos independientes mediante la directiva `fork()`. El proceso hijo actúa como el *backend*, encargándose de gestionar el tráfico de red en tiempo real a través de una arquitectura *peer-to-peer* (P2P) basada en sockets TCP/IP. El proceso padre opera como el *frontend*, administrando el bucle principal de la interfaz gráfica de usuario (GUI) y capturando las interacciones del usuario mediante la biblioteca GTK.
- **Cantidad de Tuberías:** Se implementan dos tuberías unidireccionales (*pipes*) interconectadas de forma cruzada para establecer un canal de comunicación dúplex completo (*Full-Duplex*) entre ambos procesos. Este mecanismo de comunicación entre procesos (*IPC*) es fundamental para el intercambio asíncrono de estructuras de datos tipo **Mensaje** entre la lógica de red y la interfaz visual.
- **Hilos de Ejecución (*threads*):** La concurrencia interna se gestiona mediante hilos especializados para evitar bloqueos en el flujo de datos:
  - **Proceso de Red (*Backend*):** Consta de 4 hilos distribuidos de la siguiente manera: un hilo servidor bloqueante para la escucha y aceptación de conexiones entrantes, un hilo escritor orientado a la transmisión de datos hacia el nodo remoto, un hilo lector síncrono asignado a la recepción del pipe proveniente de la GUI y un hilo escritor dedicado a canalizar los mensajes de red entrantes hacia la interfaz gráfica.
  - **Proceso Gráfico (*Frontend*):** Implementa únicamente 2 hilos asíncronos dedicados de forma exclusiva a las tareas de lectura y escritura sobre las tuberías compartidas, garantizando que la renderización de la interfaz no sufra retardos.

## 4. Desarrollo del Proyecto

### 4.1 Archivo de Cabecera (header.h)

Este archivo contiene las declaraciones de las estructuras de datos y la definición de las funciones utilizadas para la implementación del proyecto.

- **Estructura de Datos:** Se definen 2 estructuras principales; **Mensaje** y **TipoMensaje**. La primera encapsula la información relevante de cada mensaje, el id del proceso que lo envía, el contenido del mensaje, la IP y el puerto del nodo remoto, así como el tipo de mensaje que se está transmitiendo. La segunda estructura, es un enumerado que categoriza los mensajes en tres tipos; **ENVIO**, **RECIBIDO** y **SALIDA**, lo que permite una gestión eficiente de los flujos de datos y la lógica de control entre ambos procesos.
- **Funciones del Back-End:** se usan 6 funciones para el proceso de red, cada una con un rol específico en la gestión de la comunicación y la concurrencia. Estas funciones incluyen los hilos lectores y escritores tanto para la comunicación P2P como para las tuberías.

#### 4.1.1 Código

Listing 1: Archivo de declaraciones compartidas entre el Backend y el Frontend

```
1 #ifndef HEADER_H
2 #define HEADER_H
3 #include <gtk/gtk.h>
4
5 #define TAM_MAX 1024
6
7 typedef enum {
8     ENVIO,
9     RECEPCION,
10    SALIDA
11 } TipoMensaje;
12
13 typedef struct {
14     int id_emisor;
15     char texto[TAM_MAX];
16     char ip_destino[16];
17     int puerto_destino;
18     TipoMensaje tipo;
19 } Mensaje;
20
21 typedef struct {
22     int A[2]; // Interfaz <-- Backend (Backend escribe en A[1], Interfaz lee en A[0])
23     int B[2]; // Interfaz --> Backend (Interfaz escribe en B[1], Backend lee en B[0])
```

```
24 } Tuberia;
25
26 // Prototipos del Backend (conexion.c)
27 void *hilo_lector_p2p(void *arg);
28 void *hilo_escritor_p2p(void *arg);
29 void *hilo_lector_pipe(void *arg);
30 void *hilo_escritor_pipe(void *arg);
31 void conexion(Tuberia *local);
32 void enviar_msj(char *ip_destino, int puerto_destino, Mensaje msg);
33
34 // Prototipos del Frontend / Interfaz (interfaz.c)
35 void *hilo_lector_gui(void *arg);
36 void *hilo_escritor_gui(void *arg);
37 void interfaz_grafica(int argc, char *argv[], Tuberia *padre);
38
39 #endif
```

## Conclusión

Aqui va conclusion

## Referencias

- Amazon Web Services. (s. f.). ¿Qué es un cliente ligero? Amazon Web Services, Inc. <https://aws.amazon.com/es/what-is/thin-client/>
- Amazon Web Services. (s. f.-b). Qué es una CLI (interfaz de la línea de comandos)? Amazon Web Services, Inc. Amazon Web Services. (s. f.-b). Qué es una CLI (interfaz de la línea de comandos)? Amazon Web Services, Inc. <https://aws.amazon.com/es/what-is/cli>
- Velázquez, D. (2022, 22 enero). Exclusión mutua , Sistemas Operativos , WebProgramacion Consultoría Informática. WebProgramacion Consultoría Informática. <https://webprogramacion.com/exclusion-mutua/>
- Oporto Díaz, S. (s.f.). Material Adicional (SOSD – Mod 4): Concurrencia. Exclusión mutua y sincronización [PDF]. Departamento de Ciencias e Ingeniería de la Computación, Universidad Nacional del Sur. <https://cs.uns.edu.ar/~gd/soyd/clases/04-SincronizacionExtras.pdf>